

# jQuery

Internet Technology  
ITECH

BETTER JAVASCRIPT WITH JQUERY  
SELECTING, DECORATING, ENHANCING

# jQuery

- Created by John Resig
- One of the most popular JS libraries
- Simplifies client-side scripting:
  - selecting DOM elements
  - creating UI animations and effects
  - handling events
  - developing AJAX applications

# Cross Browser Compatibility

- jQuery takes a lot of the problems out of developing for multiple browsers
- Acts as a layer of abstraction over various browsers
- No more browser sniffing

# Plug-in Architecture

- jQuery creates a useful foundation for additional functionality to be added
- A wide range of specialised plug-ins have been developed since the release of jQuery for all manner of web-dev tasks (e.g., jQueryUI)

<https://www.npmjs.com/search?q=keywords:jquery-plugin>

# jQuery Syntax

- jQuery uses a basic pattern of **selecting** and **acting** on a particular DOM element and manipulating its parameters
- The selectors of CSS are reused in jQuery

`$( '#name' )`

`.text`

`( 'the new text' );`

`$( 'p' )`

`.css`

`( 'color', 'blue' );`

Select

Action

Parameters

# jQuery Selectors

```
<h1>Main Heading</h1>
<div id="section">
  <p class="text">Important.</p>
</div>
```

```
<style>
  .text {
    color: blue;
  }
  #section{
    border: 2px solid black;
  }
</style>
```

## Javascript + HTML

```
document.getElementsByClassName('text')
```

```
document.getElementById('section')
```

```
document.getElementsByTagName('h1')
```

## jQuery + CSS

```
$('.text')
```

```
$('#section')
```

```
$('h1')
```

# jQuery actions

`.text()`

`.css()`

`.hide()`

`.toggle()`

`.click()`

`.animate()`

`.ajaxSend()`

`.next()`



# jQuery parameters

- Some “actions” (methods) have no parameters

```
$("#p").show()  
$("#p").hide()
```

- Different “actions” have different parameters

```
$("#p").text("This is some text")  
$("#p").css({'font-size': '24px', 'background-color': 'black'})
```

- We may omit parameters for information retrieval

```
const currentText = $("#p").text()
```

# Clean, Consistent Markup

- jQuery reuses the pattern
- Making heavy use of anonymous functions
- Chaining functions together

# Clean, Consistent Markup

```
$(document).ready(function() {  
    alert('Hello World!');  
});
```

- `(document).ready` ensures that the code inside isn't executed until the entire page has loaded

[W3Schools Tryit Editor](#)

- Shorthand: `$()`

```
$(function() {  
    alert('Hello World!');  
});
```

# jQuery Events

| Mouse Events | Keyboard Events | Form Events | Document / Window Events |
|--------------|-----------------|-------------|--------------------------|
| click        | keypress        | submit      | load                     |
| dblclick     | keydown         | change      | resize                   |
| mouseenter   | keyup           | focus       | scroll                   |
| mouseleave   |                 | blur        | unload                   |
| hover        |                 |             |                          |

- Syntax:

`$( 'p' )`

Select

`.click`

Event

`(function() {...});`

Action

# Clean, Consistent Markup

- Here we toggle some text between visible (show) and invisible (hide)

[ToggleDisclaimer](#)

```
$('#toggleButton').click(function() {  
    if ($('#disclaimer').is(':visible')) {  
        $('#disclaimer').hide();  
    } else {  
        $('#disclaimer').show();  
    }  
});
```



# BASIC JQUERY WORKFLOW

## DEMO

# Attaching Event Handlers

Code  
Example  
*HelloWorld*

```
<html>
  <head>
    <script type="text/javascript"
      src="jquery.js"></script>

    <script type="text/javascript">
      $(document).ready(function() {
        $("a").click(function() {
          alert("Hello World!");
        });
      });

    </script>
  </head>

  <body>
    <a href="">1st Link</a><br><br>
    <a href="">2nd Link</a>
  </body>
</html>
```

# Show/Hide

```
<head>
  <script type="text/javascript" src="jquery.js">
  </script>

  <script type="text/javascript">
    $(document).ready(function() {
      $("#toggleButton").click(function() {
        $("#disclaimer").css("color", "blue");
        if ($('#disclaimer').is(':visible')) {
          $('#disclaimer').hide();
        } else {
          $('#disclaimer').show();
        }
      });
    });
  </script>
</head>

<body>
  <button id="toggleButton">Click here</button>
  <p id="disclaimer">This is a standard disclaimer.</p>
</body>
```



# Animating Elements

```
<html>
  <head>
    <script type="text/javascript"
      src="jquery.js"></script>
    <script type="text/javascript" charset="utf-8">
      $(document).ready(function() {
        $("a").hover(function() {
          $(this).animate({paddingLeft: '+=15px'}, 200);
        }, function() {
          $(this).animate({paddingLeft: '-=15px'}, 200);
        });
      });
    </script>

    <style type="text/css" media="screen">
      .bigtext { font-size: 400%; }
    </style>
  </head>

  <body>
    <a href="">1st Link</a><br><br>
    <a href="" class="bigtext">2nd Link</a>
  </body>
</html>
```

# Multiple Events

- Use the on() method

`$( 'p' )`

Select

`.on`

on

`( { click :  
function() { ... } } );`

Event: Action

```
$( "p" ).on({  
  mouseenter: function(){  
    $(this).css("background-color", "red");  
  },  
  mouseleave: function(){  
    $(this).css("background-color", "green");  
  },  
  click: function(){  
    $(this).css("background-color", "yellow");  
  }  
});
```

# Multiple Events

```
<head>
  <script src="jquery.js"></script>
  <script>
    $(document).ready(function(){
      $("p").on({
        mouseenter: function(){
          $(this).css("background-color", "red");
        },
        mouseleave: function(){
          $(this).css("background-color", "lightgreen");
        },
        click: function(){
          $(this).css("background-color", "yellow");
        }
      });
    });
  </script>
</head>

<body>
  <p>Click or move the mouse pointer over this paragraph.</p>
</body>
```

# Append to Demo

```
<head> </script>
$(document).ready(function() {
$("#btn1").click(function() {
    $("p").append("<b>Append text</b>.");
});
$("#btn2").click(function() {
    $("ol").append("<li><b>Append item</b></li>");
});
});
</script> </head>

<body>
    <p>This is a paragraph</p>
    <p>This is another paragraph</p>
<ol>
    <li>List item 1</li>
    <li>list item 2</li>
    <li>list item 3</li>
</ol>
<button id="btn1"> Append text </button>
<button id="btn2"> Append list items</button>
</body>
```

# More Demos

- Demo: jQuery UI <http://jqueryui.com/>
  - Interactions:  
<http://jqueryui.com/demos/draggable/>
  - Widgets:  
<https://jqueryui.com/button/>
  - Effects:  
<http://jqueryui.com/demos/show/>

# AJAX

Internet Technology

ITECH

# Ajax

- A key technology underlying web apps
  - **A**synchronous **J**avaScript **A**nd **X**ML
- Critically, all of the components have existed in some form since the late 1990's
  - An example of where browsers introducing non-standard features has been a positive thing

# What does Ajax do?

- Ajax **eliminates the need to reload** a web page in order to get new content from the server
  - This removes the **start-stop interaction** where a user has to wait for new pages to load.
- **Javascript** is central to AJAX
  - Improves the interactive experience in web app



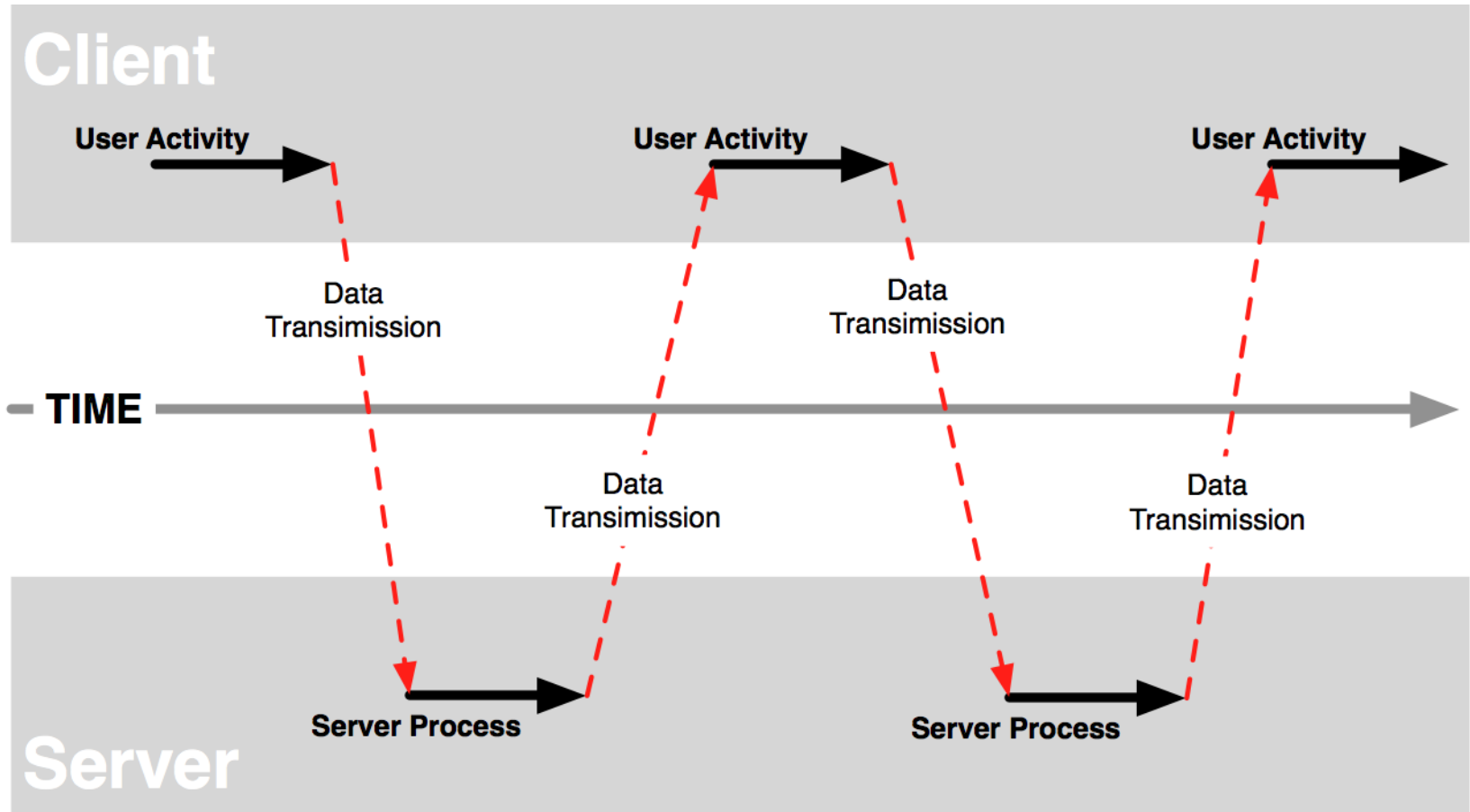
# Cheat Sheet: XML

- XML (eXtensible Markup Language)
- A **data format**: stores and transports structured information.
  - You **define your own tags** (e.g., <student>, <price>) (*unlike HTML*)
  - Focus: **meaning + structure**, not presentation (*unlike HTML*)
  - Must be **well-formed** (proper nesting, one root element, quoted attributes)

```
<Person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
  </address>
  <phoneNumber type="home">212 555-1234</phoneNumber>
</Person>
```



# THE MECHANICS



**Traditional Client/Server Synchronous Communication Model**

# AJAX Components

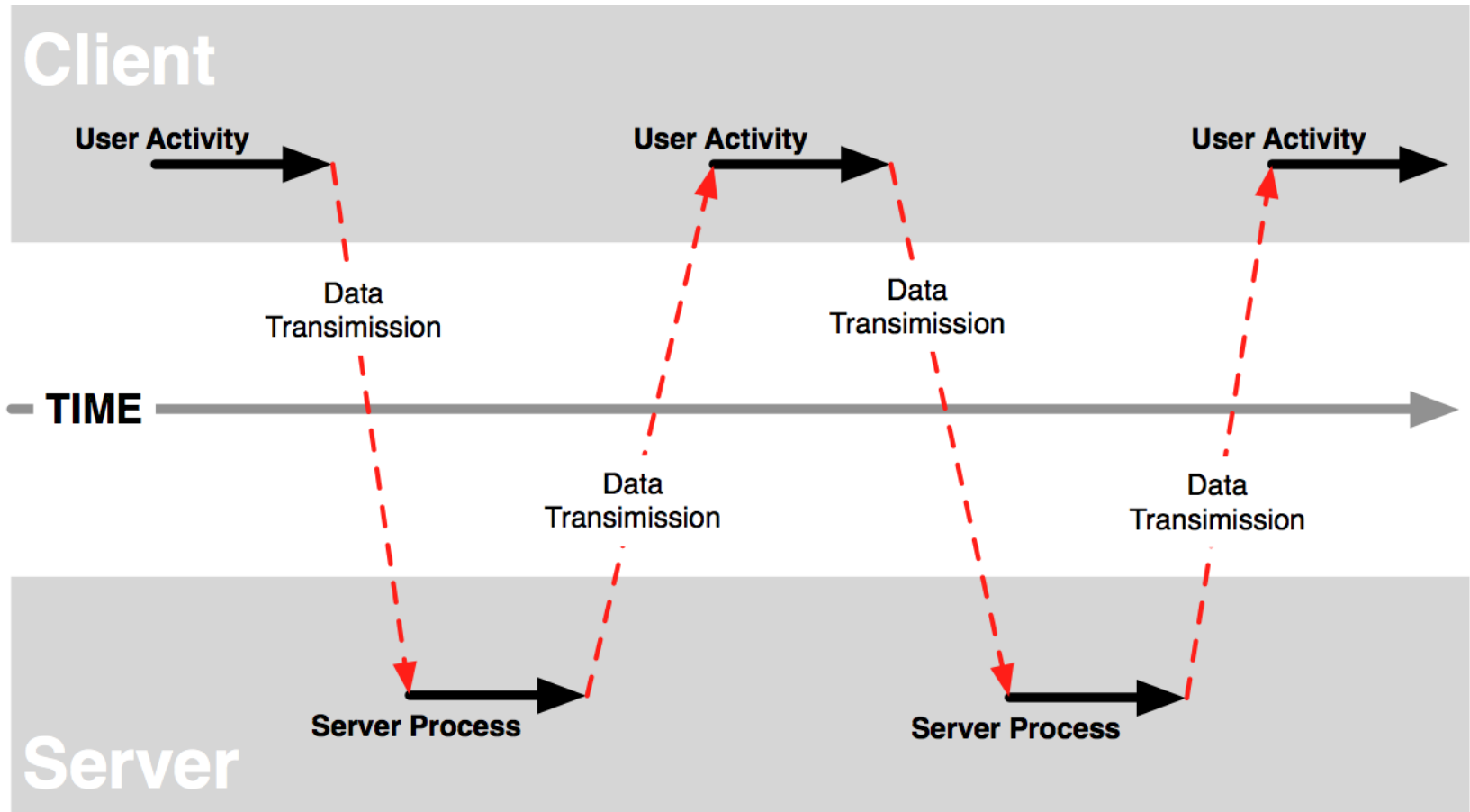
- JavaScript can manipulate the DOM of a webpage to **create, modify and remove content and style**
- JavaScript event handlers can be attached to **events generated by the user** and browser
- XML (or JSON) can **describe data** and we can access it using DOM methods
- ***So how does AJAX achieve asynchronous interaction and communication with the server?***

# XmlHttpRequest (XHR) Object

- The XHR object is the keystone of AJAX
  - Introduced by Microsoft in Internet Explorer 5
- XHR and DOM work complement each other
  - XHR fetches data
  - DOM manipulates the content of the page
- It can communicate with the server by sending HTTP requests
  - much like normal client/server communication

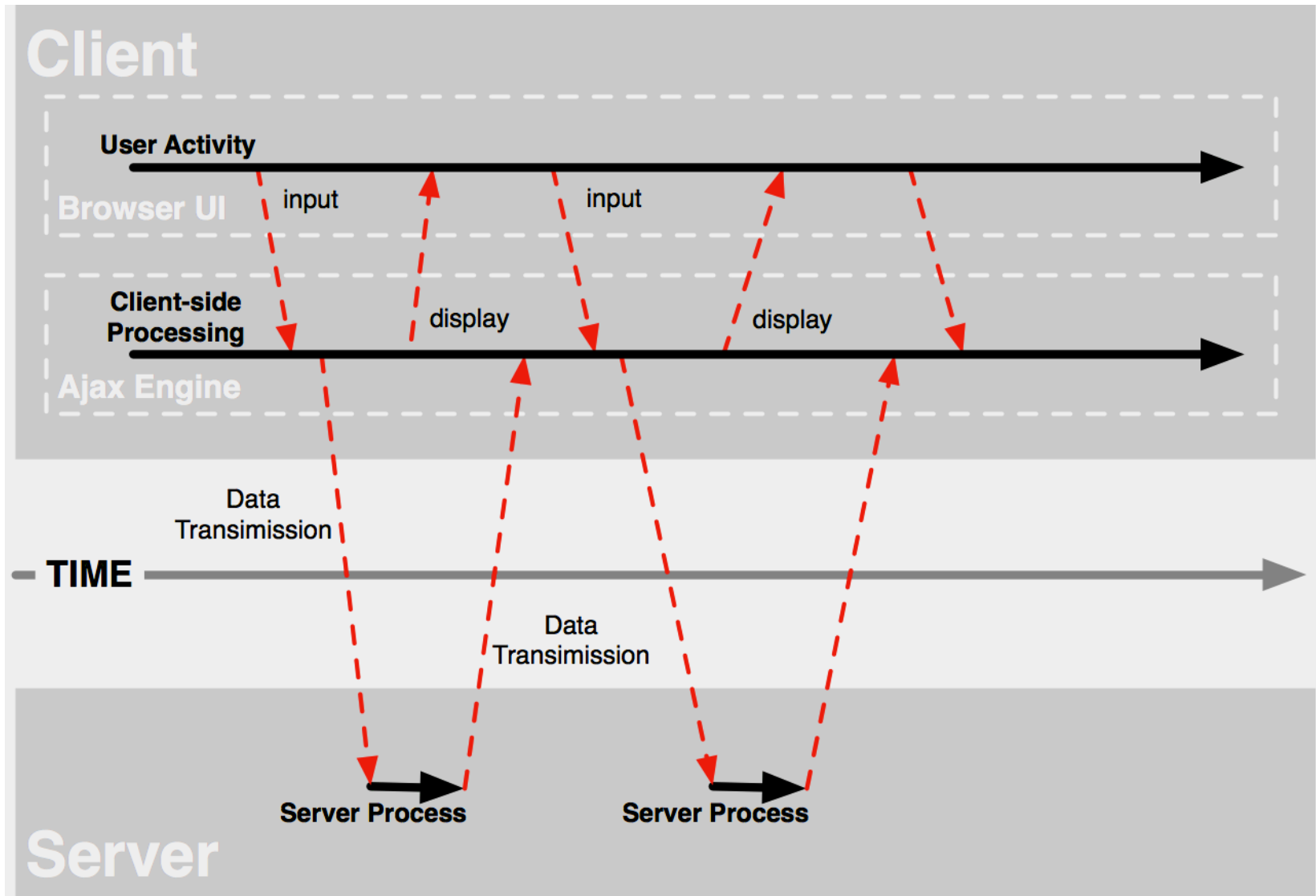
# XmlHttpRequestObject

- Independent of <form> or <a> elements for generating HTTP GET/POST requests
- It does not block script execution after sending an HTTP request
- As with **content** and **style**, JavaScript can programmatically manage **HTTP communication**



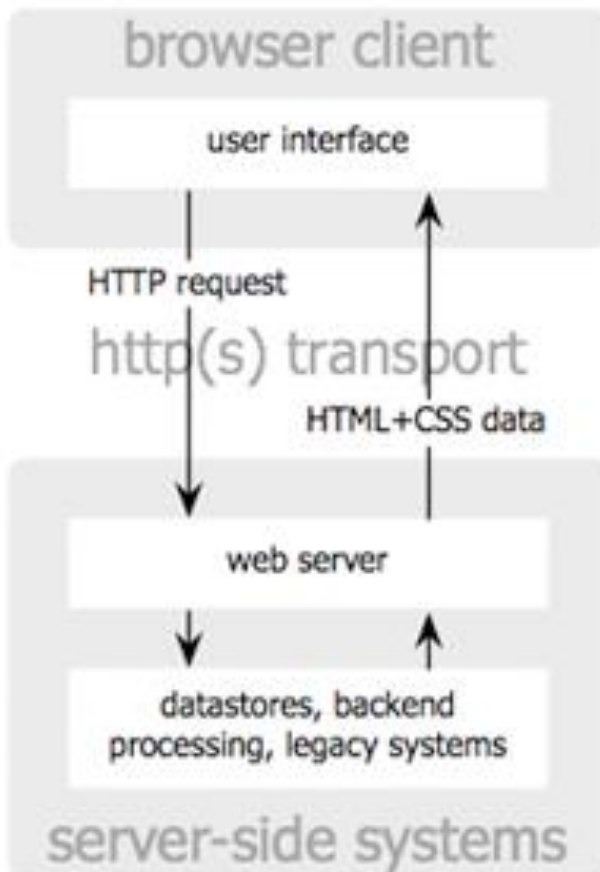
**Traditional Client/Server Synchronous Communication Model**



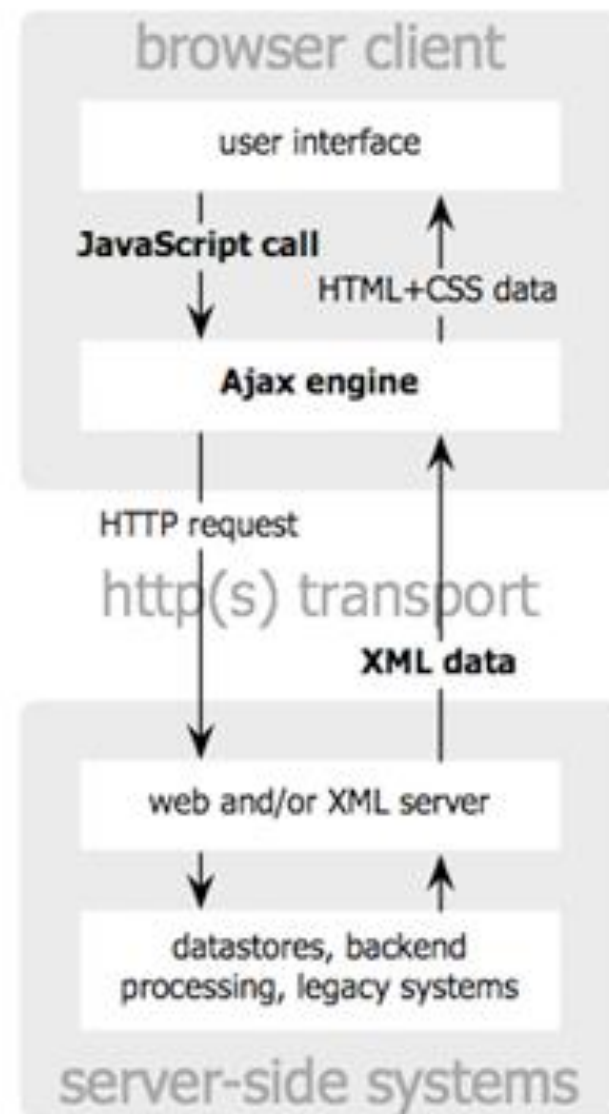


**AJAX Client/Server Asynchronous Communication Model**





**classic**  
**web application model**



**Ajax**  
**web application model**

# How AJAX works

1. An **event** occurs in a web page (the page is loaded, a button is clicked)
2. An **XMLHttpRequest** object is created by JavaScript
3. The **XMLHttpRequest** object *sends a request* to a web server
4. The server processes the request
5. The server sends a response back to the web page
6. The *response is read* by JavaScript
7. Proper action (like page update) is performed by **JavaScript**

# XmlHttpRequest Properties (1)

- **readyState** property
  - cycles through several states as it sends an xmlHttpRequest
    - 0: request not initialized
    - 1: server connection established
    - 2: request received
    - 3: processing request (has loaded enough and the user can interact with it)
    - 4: request finished and response is ready
- **onreadystatechange** property
  - accepts an EventListener value, specifying the method that the object will invoke whenever the readyState value changes
- **status** property
  - The status property represents the HTTP status code and is of type short (e.g. 200 = OK, 404 = Not Found)

# XmlHttpRequest Properties (2)

- **responseXML** property
  - represents the XML response data when the complete HTTP response has been received (when readyState is 4), and when the Content-Type header specifies the MIME (media) type as text/xml, application/xml, or ends in +xml
- **responseText** property
  - contains the text of the HTTP response received by the client
  - XML is not the only method to model data in Ajax applications. A popular alternative is JSON (JavaScript Object Notation)

# responseXML

- Simple XML to model an address book entry

```
<Person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumber type="home">212 555-1234</phoneNumber>
  <phoneNumber type="fax">646 555-4567</phoneNumber>
  <companyName />
</Person>
```

# responseText (JSON )

- Same data as before, but uses fewer characters:

```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address": {                                     // address object
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021"
  },
  "phoneNumbers": [                               // array of objects
    { "type": "home", "number": "212 555-1234" },
    { "type": "fax", "number": "646 555-4567" }
  ],
  "companyName": null
}
```

# XmlHttpRequest Methods (1)

- **open(method, url, async, user, psw)**
  - Specifies the request
    - method: the request type GET or POST
    - url: the file location
    - async: true (asynchronous) or false (synchronous)
    - user: optional user name
    - psw: optional password
- **send()**
  - Sends the request to the server
  - Used for GET requests
- **send(string)**
  - Sends the request to the server
  - Used for POST requests
- **abort()**
  - Cancels the current request

# Example

```
function loadDoc() {  
  const xhttp = new XMLHttpRequest();  
  xhttp.onreadystatechange = function() {  
    if (this.readyState == 4 && this.status == 200) {  
      document.getElementById("demo").innerHTML =  
        this.responseText;  
    }  
  };  
  xhttp.open("GET", "ajax_info.txt");  
  xhttp.send();  
}
```

[W3Schools Tryit Editor](#)



# XmlHttpRequest Methods (2)

- **setRequestHeader()**
  - Adds a name/value pair to the header to be sent
  - Can be used with POST data to specify the type of data you want to send with the send() method
- **getAllResponseHeaders()**
  - Returns all header information of a resource such as length, server-type, content-type, last-modified
- **getResponseHeader()**
  - Returns specific header information such as “Last-Modified”

**AJAX IN ACTION**

# Sending a Request

- To send a request to a server, we use the **open()** and **send()** methods of the XMLHttpRequest object:

```
xhttp.open("GET", "ajax_info.txt", true);  
xhttp.send();
```
- GET or POST?
  - GET is simpler and faster than POST, and can be used in most cases
  - However, always use POST requests when:
    - A cached file is not an option (update a file or database on the server)
    - Sending a large amount of data to the server (POST has no size limitations)
    - Sending user input (which can contain unknown characters), POST is more robust and secure than GET

# Sending a Request (cont)

- If you want to send information with the GET method, add the information to the URL

```
xhttp.open("GET", "demo_get2.asp?  
                fname=Henry&lname=Ford", true);  
xhttp.send();
```

- A simple POST request:

```
xhttp.open("POST", "demo_post.asp", true);  
xhttp.send();
```

- To POST data like an HTML form, add an HTTP header with `setRequestHeader()`. Specify the data you want to send in the `send()` method:

```
xhttp.open("POST", "ajax_test.asp", true);  
xhttp.setRequestHeader("Content-type",  
                        "application/x-www-form-urlencoded");  
xhttp.send("fname=Henry&lname=Ford");
```

# Responses from the server

- **responseText**: get the response data as a string

```
document.getElementById("demo").innerHTML = xhttp.responseText;
```

- **responseXML**: get the response data as XML data

```
xmlDoc = xhttp.responseXML;  
txt = "";  
x = xmlDoc.getElementsByTagName("ARTIST");  
for (i = 0; i < x.length; i++) {  
    txt += x[i].childNodes[0].nodeValue + "<br>";  
}  
document.getElementById("demo").innerHTML = txt;
```

```
xhttp.open("GET", "cd_catalog.xml", true);  
xhttp.send();
```

[https://www.w3schools.com/js/cd\\_catalog.xml](https://www.w3schools.com/js/cd_catalog.xml)

[https://www.w3schools.com/js/tryit.asp?filename=tryjs\\_ajax\\_xml2](https://www.w3schools.com/js/tryit.asp?filename=tryjs_ajax_xml2)

# Callback functions

- A callback function is a function passed as a parameter to another function
- If you have more than one AJAX task in a website, you should create one function for executing the XMLHttpRequest object, and one callback function for each AJAX task
- The function call should contain the URL and what function to call when the response is ready

```
loadDoc("url-1", myFunction1);  
loadDoc("url-2", myFunction2);
```

```
function loadDoc(url, cFunction) {  
    var xhttp;  
    xhttp=new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status == 200) {  
            cFunction(this);  
        }  
    }  
}
```

```
xhttp.open("GET", url, true);  
xhttp.send();  
}  
  
function myFunction1(xhttp) {  
    // action goes here  
}  
function myFunction2(xhttp) {  
    // action goes here  
}
```

# AJAX Example – HTML header

```
<!DOCTYPE html>
<html>
  <head>
    <title>AJAX example</title>
    <script type="text/javascript">
      function loadDoc() {
        var xhttp = new XMLHttpRequest();
        xhttp.onreadystatechange = function() {
          if (this.readyState == 4 && this.status == 200) {
            document.getElementById("demo").
              innerHTML = this.responseText;
          }
        };
        xhttp.open("GET", "ajax_info.txt", true);
        xhttp.send();
      }
    </script>
  </head>
```

# AJAX Example – HTML body

```
<body>
  <div id="demo">
    <h2>Let AJAX change this text</h2>
    <button type="button" onclick="loadDoc()">Change Content</button>
  </div>
  <textarea cols="50" rows="10">Enter text</textarea>
</body>
</html>
```

- Contents of ajax-info.txt:

```
<h1>AJAX</h1>
```

AJAX is not a programming language.

```
<p>
```

AJAX is a technique for accessing web servers from a web page.

```
<p>
```

AJAX stands for Asynchronous JavaScript And XML.



# jQuery AJAX

- Simplifying AJAX in web applications

```
$.ajax({type: "GET",  
        url: "greeting.php",  
        data: "name=" + name,  
        success: function(message) {  
            // do something  
        }  
});
```

# Takeaways

- JQuery is a popular JavaScript library that simplifies HTML document traversing, event handling, and animation for rapid web development.
- JQuery provides a concise syntax for selecting and manipulating DOM elements.
- AJAX is not a programming language.
- AJAX just uses a combination of:
  - A browser built-in XMLHttpRequest object (to request data from a web server)
  - JavaScript and HTML DOM (to display or use the data)